

A TRIPLE OF VECTORSTORE

Qdrant • Weaviate • Milvus

Seminar Cuối kỳ — Nhập môn Dữ liệu lớn — 5/2026

A • Lê Xuân Trí — RAG Architect

B • Nguyễn Hồ Anh Tuấn — Weaviate

C • Trần Hữu Kim Thành — Milvus

D • Trần Lê Trung Trực — Qdrant

AGENDA

01 OVERVIEW

— Tại sao cần Vector DB? [6']

02 ARCHITECTURE

— Kiến trúc 3 công cụ [21.5']

03 EVALUATION

— Benchmark & kết quả [6']

04 DEMO

— Thực chiến [7']

Speakers: A · D · B · C

WHY VECTOR DATABASE?



Knowledge Cutoff:
LLM chỉ biết dữ liệu cũ.



Hallucination:
Bịa thông tin khi thiếu context.



No Private Data:
Không truy cập tài liệu nội bộ.

Giải pháp RAG:

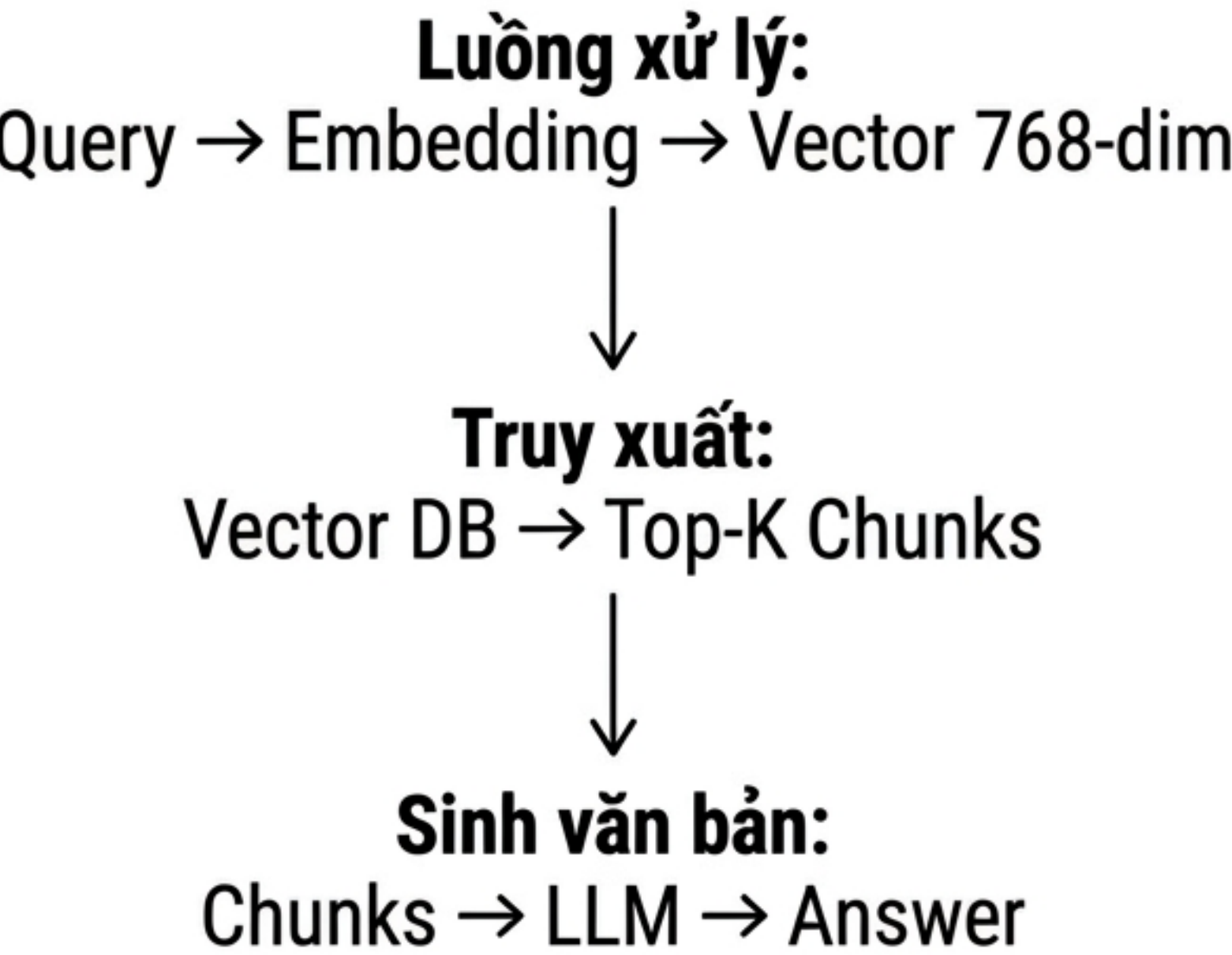
Tìm tài liệu → Bổ sung context → LLM trả lời



Vector DB:

Đóng vai trò bộ nhớ ngữ nghĩa của RAG.

RAG PIPELINE



Tiêu chí	SQL Truyền thống	Vector DB
Tìm kiếm	Exact match (WHERE)	Similarity (ANN)
Dữ liệu	Rows / Documents	Vectors 768+ chiều
Ý nghĩa	Theo từ khóa	Theo ngữ nghĩa

THE BIG THREE

Qdrant

Rust · 2021

Performance first

Filtered HNSW,
1 container.

Weaviate

Go · 2019

AI-native all-in-one

Hybrid BM25F,
Modules.

Milvus

C++ & Go · 2019

Distributed enterprise

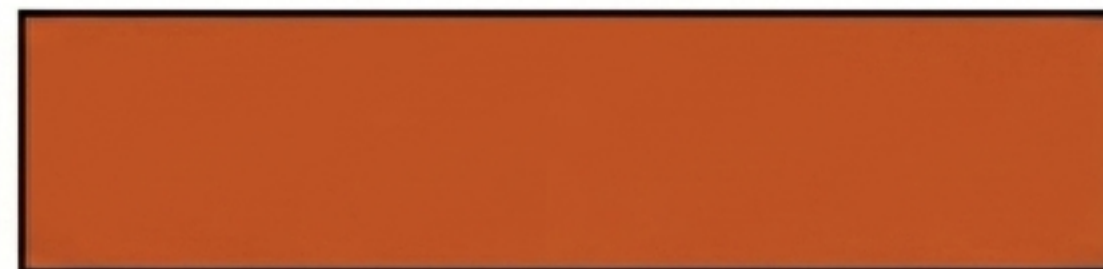
Billion-scale,
3+ containers.

GITHUB STARS (5/2026)

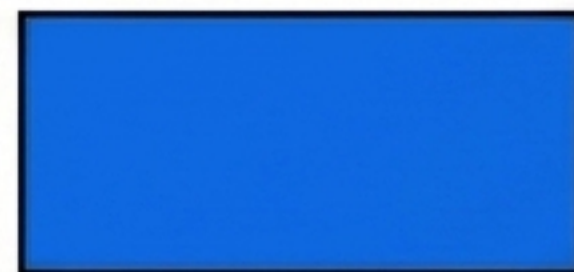
Milvus: 44,400+ (Linux Foundation)



Qdrant: 31,400+ (Tăng nhanh nhất)



Weaviate: 16,200+ (Tập trung DX)



Lưu ý: Stars $\neq$ Chất lượng thực tế.

OPEN-SOURCE LANDSCAPE

Tiêu chí	Qdrant	Weaviate	Milvus
License	Apache 2.0	BSD 3	Apache 2.0
Quản trị	Qdrant GmbH	Weaviate B.V.	LF AI
Self-host	✅ Docker	✅ Docker	✅ Operator

- Milvus:
- Dự án tốt nghiệp của LF AI Foundation.
- Quản trị độc lập, cùng cấp ONNX.
- Qdrant & Weaviate do startup điều hành.

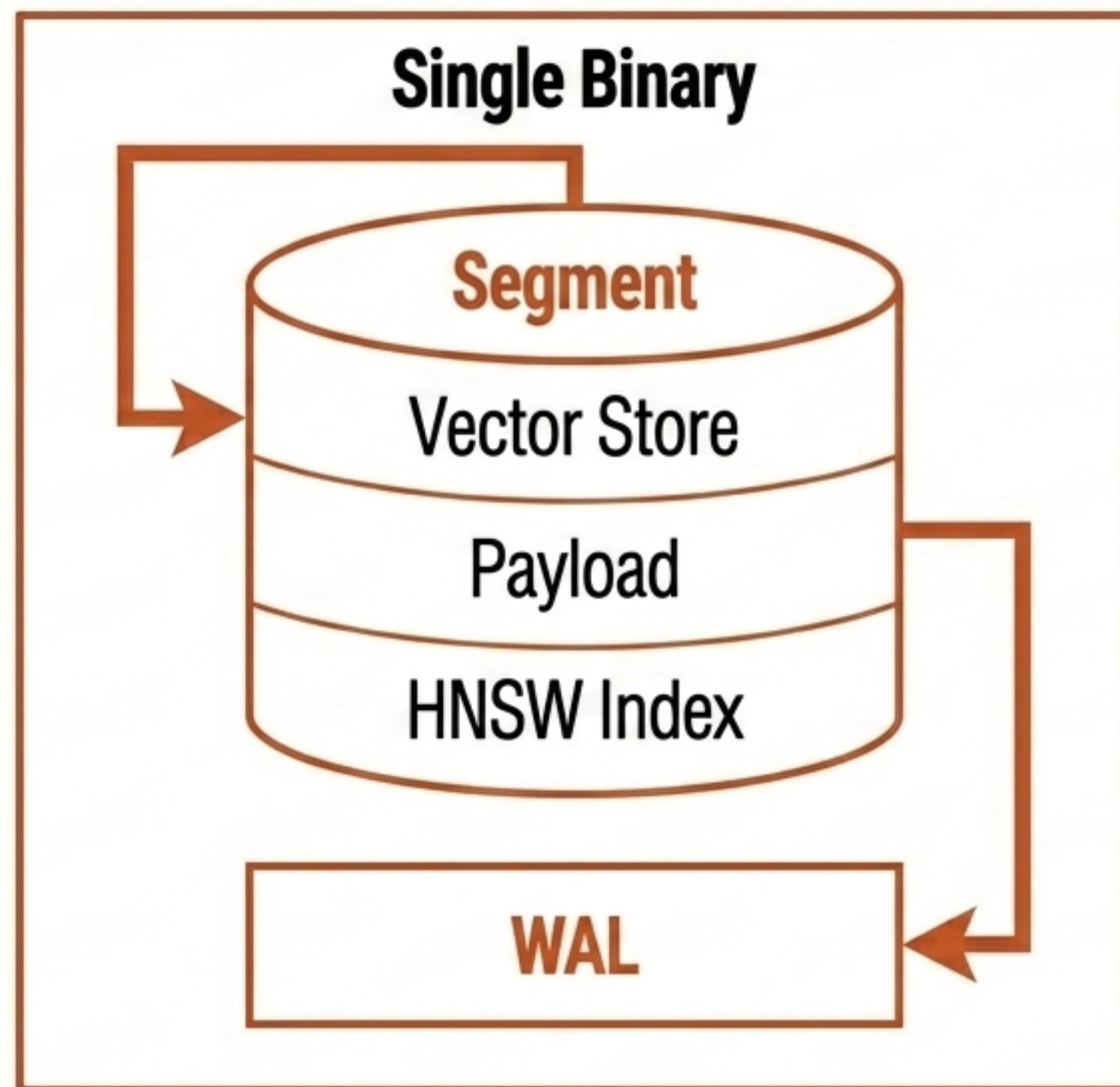
QDRANT — RUST ENGINE

- **Single Binary (Rust):**
 - Zero GC pause.
- **Segment:**
 - Đơn vị lưu trữ tự trị.
 - Chứa Vector Store + Payload + HNSW Index.
- **WAL:**
 - Đảm bảo tính toàn vẹn dữ liệu.

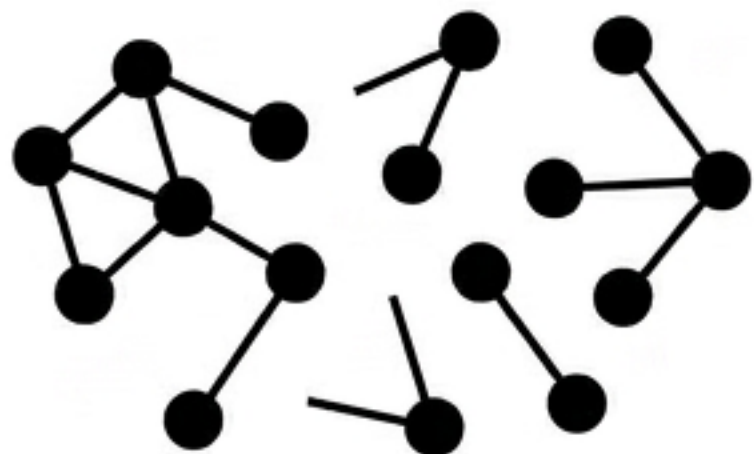
Flow: REST/gRPC → Query Planner → Segments.

Storage: Memmap + WAL.

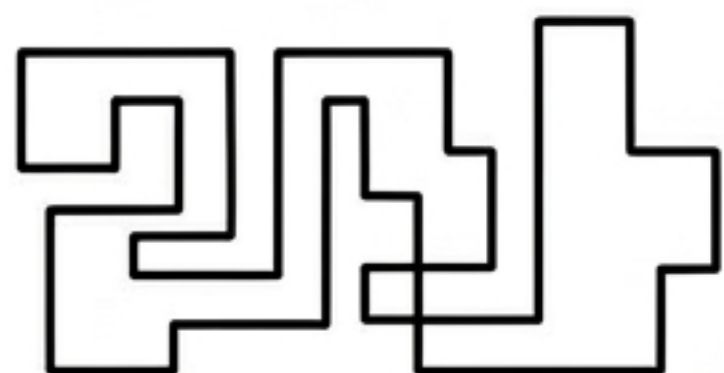
Key Stats: 1 container, ~79MB RAM. Ready ngay.



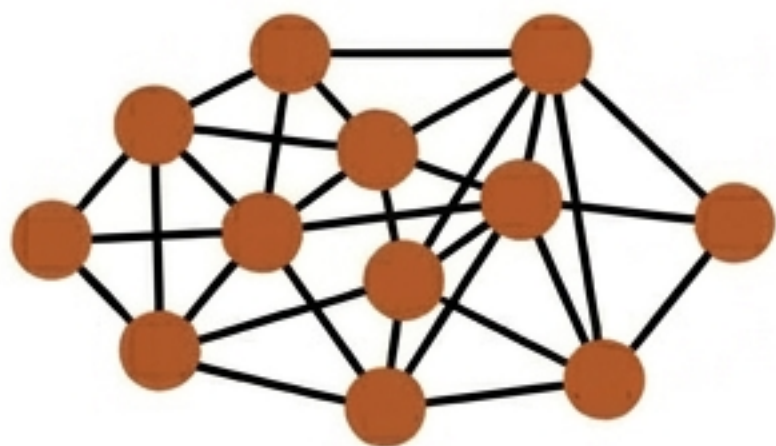
FILTERABLE HNSW & ACORN



Pre-filter: Đồ thị phân mảnh → Recall giảm.



Post-filter: Thiếu kết quả → Latency tăng.



In-graph (Qdrant): Lọc tại mỗi node → Giữ recall.

Thuật toán ACORN
(Adaptive):

- Match nhiều → Duyệt graph + skip.
- Match ít → Dùng brute-force trực tiếp.
- Planner tự động tối ưu.

FILTER STRATEGY COMPARISON

Chiến lược	Recall	Latency	Dùng bởi
Pre-filter	↓ Giảm	Thấp	Weaviate
Post-filter	Giữ	↑ Cao	Đa số
In-graph	✓ Giữ	✓ Thấp	Qdrant

- In-graph kết hợp ưu điểm tốt nhất.
- Lọc nhanh hơn query thuần túy.
- Loại candidate sớm → Giảm tính toán distance.